DATA MODELING

What is Data Modeling?

- **Definition:** The process of defining how tables interact with each other inside Power BI.
- **Goal:** To connect isolated tables using relationships so that filters applied to one table automatically flow to others.
- Similar to performing JOINs in SQL, but done visually in Power BI's Model View.
- Result: Transforms separate spreadsheets into one intelligent, connected system.
- **Analogy (Company Departments):**

  Orders: Sales department (transactions).

  Returns: Customer service department (returned orders).

  People: HR department (sales rep per region).

  A relationship is the link between these departments, enabling instant cross-departmental queries (e.g., "Which sales representative had the most returns?").

1. Star Schema

- The foundation for structuring data in Power BI.
- Comprises two primary table types: **Fact Tables** and **Dimension Tables**.

1.1 Fact vs. Dimension Table Distinction

Property	Fact Table	Dimension Table
Contains	Numbers you calculate (Measures)	Descriptions / Labels (Attributes)

Examples	Sales, Profit, Quantity	Customer Name, Product, Region
Rows	Many (millions possible)	Few (hundreds to thousands); One per entity
Changes	Constantly(new transactions)	Rarely
Purpose	What you calculate	What you filter and group by
Analogy	The receipt	The menu / address book

Superstore Example: Orders is the **Fact Table** (transactions with numbers). Returns, People, etc., are **Dimension Tables** (describe *who, what, where*).

Why Use a Star Schema?

- **Performance:** VertiPaq engine compresses separated text values (e.g., "Technology") instantly, improving efficiency over flat tables.
- **Accuracy:** Ensures DAX measures (like distinct customer count) have correct filter context, preventing double-counting.
- **Maintainability:** Renaming a product category requires updating only 1 row in the Products dimension, minimizing inconsistency risk.
- **Scalability:** New data sources can be added via new relationships without restructuring the entire dataset.

1.2 Relationships and Cardinality

- **What is a Relationship?**

  A link between two tables based on a matching column.Uses a **Primary Key (PK)** → **Foreign Key (FK)** pattern (visual SQL JOINs).

Term	Meaning	Example
Primary Key (PK)	Unique ID in a dimension table (appears exactly once)	Customer ID in Customer table
Foreign Key (FK)	The same ID in the fact table (appears many times)	Customer ID in Orders table
Relationship	The link between the two keys	Orders[Customer ID] → Customer[Customer ID]

- **What is Cardinality ?**

  Defines the numeric relationship between the tables.

Type	Symbol	Meaning	Rule of Thumb
One-to-Many	1:*	Most common. One row in dimension matches many rows in fact	Most Common;(Dim → Fact),Location→ Orders
One-to-One	1:1	Rare. One row matches exactly one row	Employee ↔ Employee Details
Many-to-Many	*:*	Avoid if possible; use a bridge table instead	Orders ↔ Returns (needs bridge)

Building Relationships (Step-by-Step)

Step 1**:** Go to Model View: Click the Model View icon (third icon on left sidebar, Ctrl+3).

Step 2: Drag to Create: Click and drag a column (e.g., Order ID from Orders) and drop it onto the matching column in the target table (e.g., Order ID in Returns).

Step 3: Repeat: Connect all relevant tables (e.g., Drag Region from Orders to Region in People).

Step 4: Check Properties (Double-Click Line):

Step 4.1: Cardinality: Should be One-to-Many (1:*).

Step 4.2: Cross Filter Direction: Should be Single (→).

Step 4.3: Active: Should be checked (green tick).

- Verify Diagram: Check for solid lines with 1 on the dimension side and * on the fact side.

1.3 Filter Direction — The Critical Flow

- **Concept:** Controls the direction filters travel between tables.
- Rule: Filters should always flow from Dimension tables INTO the Fact table (one-way flow).

Direction	Symbol	Meaning	When to Use
Single	→	Filters flow one way only (dim → fact)	Default. Use always in star schema
Both	↔	Filters flow both ways	Rarely needed. Can cause ambiguity anslow performance

- **Benefits of Single Direction:**
- Prevents circular filter dependencies.
- Avoids unexpected cross-filtering between dimension tables.
- Maintains report performance on large datasets.
- Produces correct aggregations in DAX measures.

Note: Both Direction

Only used in specific many-to-many scenarios, activated deliberately via USERELATIONSHIP() in DAX—never as a default setting.

2. Detailed Explanation of Data Modelling I did using Superstore dataset

2.1 Data Source

Property	Detail
Source Type	Single Excel Workbook (.xlsx)
Original Sheets	Orders, Returns, People
Primary Table	Orders (9,994 rows, 21 columns)
Date Range	January 2014 — December 2018
Connection Mode	Import

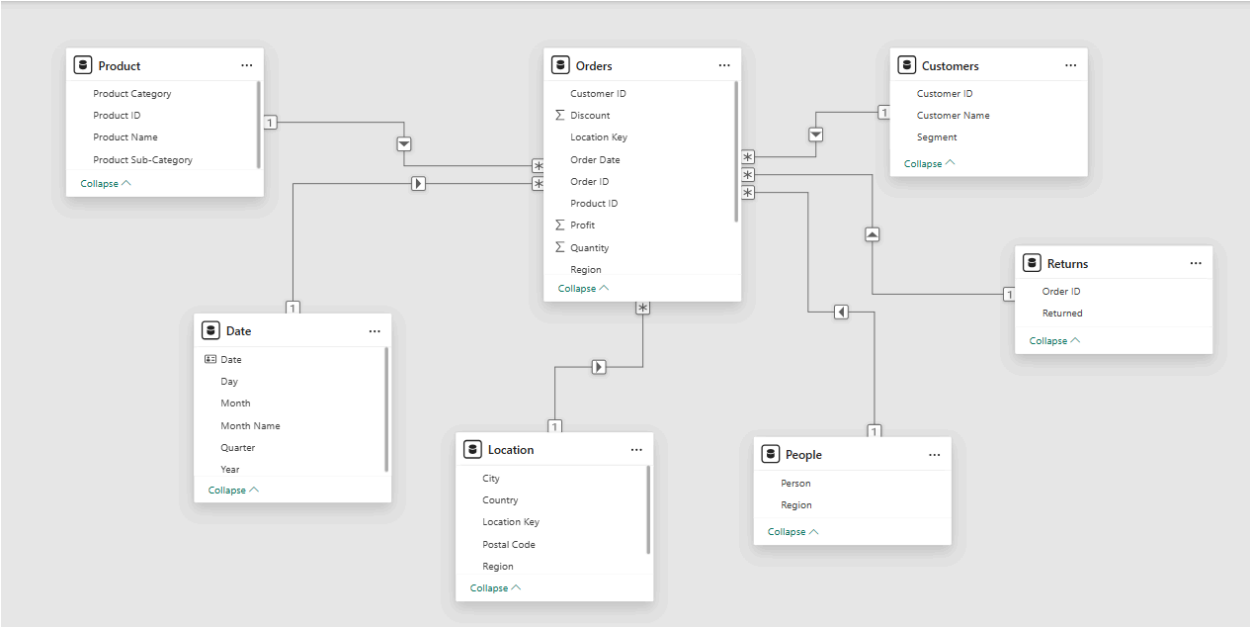
Original Orders Table Columns (21 columns)

Column	Data Type	Description
Row ID	Whole Number	Unique row identifier
Order ID	Text	Transaction identifier
Order Date	Date	Date order was placed
Ship Date	Date	Date order was shipped
Ship Mode	Text	Shipping method
Customer ID	Text	Unique customer identifier

Customer Name	Text	Full name of customer
Segment	Text	Customer segment (Consumer/Corporate/Home Office)
Country	Text	Shipping country
City	Text	Shipping city
State	Text	Shipping state
Postal Code	Whole Number	Shipping postal code
Region	Text	Geographic sales region
Product ID	Text	Unique product identifier
Category	Text	Product category
Sub-Category	Text	Product sub-category
Product Name	Text	Full product name
Sales	Decimal	Revenue from order line
Quantity	Whole Number	Units ordered
Discount	Decimal	Discount applied
Profit	Decimal	Profit from order line

2.2 Star Schema Design

2.2.1 Schema Diagram



2.2.2 Table Classification

Table	Type	Role
Orders	Fact Table	Central table containing all measurable transaction data
Customer	Dimension Table	Describes who placed the order
Products	Dimension Table	Describes what was ordered
Location	Dimension Table	Describes where the order was shipped
Date	Dimension Table	Describes when the order was placed

People	Dimension Table	Describes which sales rep covers which region
Returns	Bridge Table	Connects returned orders to the fact table

2.3 Tables Understanding

2.3.1 Orders (Fact Table)

The central fact table. Contains only foreign keys, date columns, and numeric measures. All descriptive attributes have been removed and placed in their respective dimension tables.

Column	Data Type	Role	Links To
Order ID	Text	Transaction identifier	Returns[Order ID]
Order Date	Date	Foreign key	Date[Date]
Ship Date	Date	Shipping analysis	—
Ship Mode	Text	Shipping method	—
Customer ID	Text	Foreign key	Customer[Customer ID]
Product ID	Text	Foreign key	Products[Product ID]
Region	Text	Foreign key	People[Region]

Note: On Ship Mode

Ship Mode is a descriptive attribute and ideally belongs in a Shipment dimension table. It is retained in the fact table at this stage as an acceptable simplification. A future improvement would be to extract it into a dedicated Shipment dimension.

2.3.2 Customer (Dimension Table)

Describes the customer who placed the order.

Column	Data Type	Role
Customer ID	Text	Primary Key — unique per customer
Customer Name	Text	Full name
Segment	Text	Consumer / Corporate / Home Office

Primary Key: Customer ID (each value appears exactly once)

2.3.3 Products (Dimension Table)

Describes the product that was ordered.

Column	Data Type	Role
Product ID	Text	Primary Key — unique per product
Product Name	Text	Full product name
Category	Text	Technology / Furniture / Office Supplies
Sub-Category	Text	Phones, Chairs, Binders, etc.

Primary Key: Product ID (each value appears exactly once)

2.3.4 Location (Dimension Table)

Describes the shipping destination of the order.

Column	Data Type	Role
Location Key	Text	Primary Key — composite key (City + "-" + Postal Code)
City	Text	Shipping city
State	Text	Shipping state
Country	Text	Shipping country
Region	Text	Geographic region
Postal Code	Text	Shipping postal code

Primary Key: Location Key (composite)

Why composite key: Raw Postal Code is not unique — the same city can have multiple postal codes, and the same postal code can appear with inconsistently entered city names. Combining City and Postal Code creates a reliable unique identifier.

2.3.5 Date (Dimension Table)

A complete calendar table with no gaps. Built independently — not extracted from Orders.

Column	Data Type	Role
Date	Date	Primary Key — every calendar date, no gaps
Year	Whole Number	Calendar year

Month	Whole Number	Month number (1–12)
Month Name	Text	January, February, etc.
Quarter	Text	Q1, Q2, Q3, Q4

Primary Key: Date

Range: 1 January 2014 — 31 December 2018 (1,826 continuous days)

Marked as Date Table: Yes — required for time intelligence DAX functions

Note: This table was built using Power Query's List.Dates() function to ensure every single calendar date is present. Using Order Date from the Orders table directly would create gaps on days with no orders, breaking all time intelligence DAX functions.

2.3.6 People (Dimension Table)

Describes which sales representative is responsible for each geographic region.

Column	Data Type	Role
Region	Text	Primary Key — unique per region
Person	Text	Sales representative name

Primary Key: Region

2.3.7 Returns (Bridge Table)

A bridge table connecting returned orders to the fact table. It is not a dimension table because it does not describe any entity — it only records whether an order was returned.

Column	Data Type	Role
Order ID	Text	Foreign key → Orders[Order ID]

Returned	Text	"Yes" for all rows (only returned orders are listed)

Type: Bridge table (resolves the many-to-many nature of returns)
Why not a dimension: Has no descriptive attributes of its own. Contains only an Order ID and a status flag.

2.4. Power Query Transformations

2.4.1 Method: Duplicate vs Reference

All dimension tables were created using **Duplicate**, not Reference.

Feature	Duplicate	Reference
What it creates	An independent copy of the query and all its Applied Steps at that moment	A live pointer — the new query's first step IS the original query
On source refresh	Does NOT update — stays frozen at the time of duplication	Updates automatically — all changes to the source flow through
Use case	When you need an isolated snapshot intentionally	Always — when building dimension tables from a fact table
For Superstore	Not recommended for dimension tables	Used for Customers, Products, Location dimensions

Why Duplicate for dimensions: When you remove Customer Name from Orders, it should remain in the Customer table. With Reference, removing a column from Orders removes it from every dependent query. With Duplicate, each table manages its own columns independently.

2.4.2 Orders Table Transformations

Steps applied in Power Query:

Step 1: Loaded from Excel source

Step 2: Verified data types for all columns

Step 3: Added Location Key custom column: = [City] & "-" & Text.From([Postal Code])

Step 4: Removed columns now living in dimension tables: Customer Name, Segment, Product Name, Category, Sub-Category, City, State, Country, Postal Code

Step 5: Verified remaining columns: Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Product ID, Location Key, Region, Sales, Quantity, Discount, Profit

2.4.3 Customer Table Transformations

Step 1: Duplicated Orders query → renamed to Customer

Step 2: Removed all columns except: Customer ID, Customer Name, Segment

Step 3: Removed duplicates on Customer ID

Step 4: Verified row count matches expected unique customer count

2.4.4 Products Table Transformations

Step 1 : Duplicated Orders query → renamed to Products

Step 2 : Removed all columns except: Product ID, Product Name, Category, Sub-Category

Step 3 : Removed duplicates on Product ID

Step 4 : Verified row count matches expected unique product count

2.4.5 Location Table Transformations

Step 1: Duplicated Orders query → renamed to Location

Step 2: Removed all columns except: City, State, Country, Postal Code

Step 3: Added Location Key custom column: = [City] & "-" & Text.From([Postal Code])

Step 4: Removed duplicates on Location Key

Step 5: Verified no duplicate Location Keys remain

2.4.6 Date Table Construction

Built from scratch using Power Query — not extracted from Orders:

= List.Dates(#date(2014,1,1), 1826, #duration(1,0,0,0))

Step 1: New Source → Blank Query

Step 2: Entered formula above in formula bar

Step 3: Right-click list → Convert to Table

Step 4: Renamed column to Date

Step 5: Changed data type to Date

Step 6: Added Year column via Add Column → Date → Year

Step 7: Added Month column via Add Column → Date → Month

Step 8: Added Month Name via Add Column → Date → Month Name

Step 9: Added Quarter via Add Column → Date → Quarter

Step 10: Renamed query to Date ->Close and Apply

Step 11: In Power BI Desktop: right-click Date table → Mark as Date Table → selected Date column

2.4.7 People Table Transformations

Step 1 : Loaded directly from People sheet in Excel source

Step 2 : Verified columns: Region, Person

Step 3 : Verified Region values match exactly with Orders[Region] — same capitalisation, no trailing spaces

Step 4 : No duplicates on Region

2.4.8 Returns Table Transformations

Step 1: Loaded directly from Returns sheet in Excel source

Step 2: Verified columns: Order ID, Returned

Step 3: No transformation required

2.5. Data Model Relationships

2.5.1 Relationship Summary

S.No	From (One side)	To (Many side)	Key Column	Cardinality	Filter Direction	Active
1	Customer[Customer ID]	Orders[Customer ID]	Customer ID	One-to-Many (1:*)	Single	Yes
2	Products[Product ID]	Orders[Product ID]	Product ID	One-to-Many (1:*)	Single	Yes
3	Location[Location Key]	Orders[Location Key]	Location Key	One-to-Many (1:*)	Single	Yes
4	Date[Date]	Orders[Order Date]	Date	One-to-Many (1:*)	Single	Yes
5	People[Region]	Orders[Region]	Region	One-to-Many(1:*)	Single	Yes
6	Returns[Order ID]	Orders[Order ID]	Order ID	One-to-Many (1:*)	Single	Yes

3. Key Design Decisions

Decision 1: Composite Location Key

Problem: Postal Code alone is not a reliable primary key for the Location table. The same city can have multiple postal codes, and the same postal code can appear with inconsistent city name entries.

Solution: Created a composite key combining City and Postal Code:

Location Key = [City] & "-" & Text.From([Postal Code])

Example: New York City-10001 is unique and unambiguous.

Decision 2: Dedicated Date Table

Problem: Using Order Date directly from Orders creates a date column with gaps — days with no orders are missing. Power BI's time intelligence functions require a contiguous date table.

Solution: Built an independent Date table using List.Dates() covering every calendar day from 2014 to 2018. Marked as Date Table in Power BI Desktop.

Decision 3: People Connected Directly to Orders

Problem: People table contains Region. Location table also contains Region. An initial attempt connected People → Location → Orders as a chain.

Why this is wrong: Power BI does not automatically propagate filters through relationship chains the way SQL joins do. Chained relationships create ambiguity and Power BI deactivates one relationship (shown as dashed line in Model View).

Solution: People connects directly to Orders via Region. Location connects directly to Orders via Location Key. No relationship between People and Location.

Decision 4: Duplicate over Reference for Dimensions

Problem: Using Reference to create dimension tables makes them dependent on the Orders query's final output. Removing a column from Orders removes it from all Reference-based dimension tables.

Solution: Used Duplicate for all dimension tables. Each dimension manages its own column list independently. Removing Customer Name from Orders has zero effect on the Customer dimension table.

4. Mistakes Made & Lessons Learned

These are real mistakes made during the building of this model — documented for learning purposes.

Mistake 1: Using Reference Instead of Duplicate

What happened: Initially created all dimension tables using Reference from Orders. When attempting to remove Customer Name from Orders, it disappeared from the Customer table too.

Why it happened: Misunderstood the difference between Reference and Duplicate. Reference creates a dependent query that inherits the source's final output. Duplicate creates an independent copy.

Fix: Deleted all Reference-based dimension queries. Rebuilt every dimension using Duplicate.

Lesson: Use Reference only when you intentionally want downstream tables to reflect upstream changes. Use Duplicate when you need independent column control.

Mistake 2: Relying on Auto-Hierarchy Instead of Building a Date Table

What happened: Noticed that Order Date in Power BI automatically showed Year → Quarter → Month → Day hierarchy and assumed this was sufficient.

Why it was wrong: The auto-hierarchy is a display feature only. It is not a marked Date Table and does not contain continuous dates. Every time intelligence DAX function requires a proper marked Date Table with no gaps.

Fix: Built a dedicated Date table using List.Dates() in Power Query and marked it as Date Table in Power BI Desktop.

Lesson: Never confuse Power BI's auto-generated date hierarchy with an actual Date dimension table.

Mistake 3: Chaining People → Location → Orders

What happened: Connected People to Location via Region, expecting filters to flow through to Orders automatically.

Why it was wrong: Power BI relationship filters do not chain automatically. This created an ambiguous filter path and Power BI deactivated one of the relationships.

Fix: Connected People directly to Orders via Region. Removed the People → Location relationship entirely.

Lesson: Every dimension table should have a direct relationship to the fact table. Avoid indirect chains.

Mistake 4: Hiding Columns Instead of Removing Them

What happened: When I need to remove Customer Name and Segment from the Orders fact table, initially hid the columns instead of removing them.Because I used reference instead of duplicate.

Why it was wrong: Hiding is cosmetic. The columns still exist in the table, consuming memory and creating risk of incorrect relationships or accidental use in DAX.

Fix: Removed the columns properly in Power Query using Remove Columns step.

Lesson: Hide columns only as a last resort for UX purposes. Always remove columns that genuinely don't belong in a table.

Model Verification Checklist

Before proceeding to DAX, I verified the following(Refer it):

Check	Expected Result	Status
No dashed lines in Model View	All 6 relationships are active (solid lines)	Pass
Date table marked	Calendar icon appears next to Date table	Pass

Orders contains no descriptive columns	Only IDs, dates, Ship Mode, and measures	Pass
Cross-table visuals work	Customer Name + Sales shows correct individual totals	Pass
Category + Profit visual works	Correct profit per product category	Pass
Region + Sales visual works	Correct sales per region	Pass

What This Model Enables

With this star schema in place, the following analysis are now possible:

Business Questions This Model Can Answer

Question	Tables Involved
Which customer segment is most profitable?	Orders + Customer
Which product sub-category has the highest discount rate?	Orders + Products
Which sales rep's region had the most returns?	Orders + People + Returns
What is the month-over-month sales growth?	Orders + Date
Which city has the lowest profit margin?	Orders + Location

Which products were returned most frequently?	Orders + Products + Returns
How does this year's sales compare to last year?	Orders + Date (time intelligence)

Power Query Formula Used

1. **Location Key (Custom Column)**

   = [City] & "-" & Text.From([Postal Code])

2. **Date Table Generation**

   = List.Dates(#date(2014,1,1), 1826, #duration(1,0,0,0))

3. **Remove Duplicates**

Applied via: Home → Remove Rows → Remove Duplicates (after selecting the primary key column)